UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniería de Software 1 – Sección 20

Catedrático: Erick Francisco Marroquín Rodríguez

Corte 3 del proyecto. Análisis y diseño de aplicación para la supervisión de personal y gestión de activos de Teleprogreso, S.A.

Harry Mendez – 24089

Juan Gualim – 24852

Biancka Raxón – 24960

Joel Nerio – 24253

Wilson Peña – 24760

Diego Quixchan – 24903

GUATEMALA, marzo del 2026

Resumen

Se detecto que el tema de control ya sea del tiempo o recursos en las empresas, puede ser una de las causas más probables por las cuales estas no logran completar objetivos o tareas, que se habían establecido con anterioridad. Por lo que la razón del proyecto es el poder ayudar a la empresa Teleprogreso S.A. a poder desarrollar un software diseñado para poder llevar un control de sus empleados y como estos utilizan los activos de la empresa, ya que muchas veces estos dentro de su horario laboral optan por hacer actividades seculares y personales usando muchas veces los activos empresariales, resultando así en tiempo perdido y en recursos mal gestionados que a la larga puede repercutir en la eficiencia y calidad de la misma empresa. Por lo que se tiene como objetivo el poder controlar estos escenarios, para así poder tener una mejor gestión empresarial y una eficiencia de tiempo a través del control de personal.

Introducción

Teleprogreso S.A. es una empresa proveedora de servicios de internet por fibra óptica y televisión por cable (ISP) con sede en Fraijanes, Guatemala. Con una fuerza laboral de 24

empleados, la organización gestiona procesos críticos que incluyen instalaciones nuevas, reparación de fallas técnicas, cobros en ruta y recuperación de equipos domiciliares. A pesar de su presencia activa en el mercado local, la entidad enfrenta retos significativos derivados de la falta de un sistema automatizado para la supervisión de su personal de campo y la administración de sus activos físicos.

Actualmente, el control de la jornada laboral y el uso de recursos como vehículos y herramientas especializadas se realiza de forma manual o a través de comunicaciones informales por aplicaciones de mensajería. Esta desarticulación operativa genera una visibilidad limitada de las actividades en tiempo real, dependencia excesiva de llamadas constantes para la coordinación de tareas y una gestión ineficiente de los activos, lo que eventualmente impacta en la calidad del servicio al cliente y en los costos operativos de la empresa.

Ante esta problemática, surge la propuesta de desarrollar una plataforma integral de supervisión y gestión que permita centralizar la comunicación operativa y automatizar el seguimiento de los técnicos en campo. A través de módulos de geolocalización, control de asistencia y registro de inventario, el sistema busca transformar los procesos actuales en flujos de trabajo estructurados y transparentes. El presente informe documenta la transición desde la ideación hasta la modelación detallada del sistema, estableciendo las bases técnicas necesarias para su implementación futura.

Objetivos

Objetivo general

Desarrollar un sistema de supervisión y control que permita a Teleprogreso S.A. llevar un mejor seguimiento de los horarios laborales de sus empleados y del uso de los activos de la empresa.

Objetivos específicos

- Implementar un módulo de control de asistencia que permita registrar entrada, salida y tiempos de descanso de los empleados, apoyado en geolocalización cuando sea posible.
- Desarrollar un sistema básico de gestión de activos que permita registrar el uso de vehículos y herramientas asignados a cada empleado.

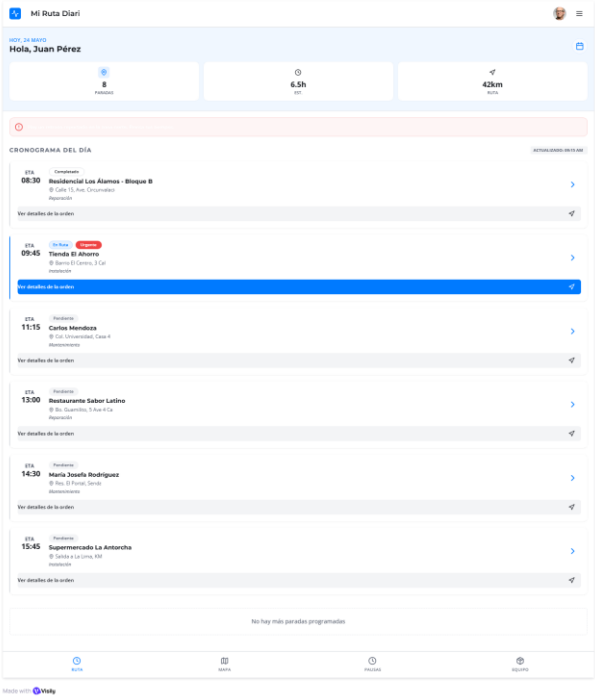
- Crear un panel de visualización que facilite a los supervisores el seguimiento del estado de las rutas, técnicos en campo y activos disponibles.
- Facilitar el reporte de incidencias por parte de los técnicos, permitiendo registrar daños o problemas de manera sencilla y ordenada.
- Apoyar la asignación y reasignación de rutas de trabajo mediante información clara sobre disponibilidad y ubicación del personal.

Design Thinking

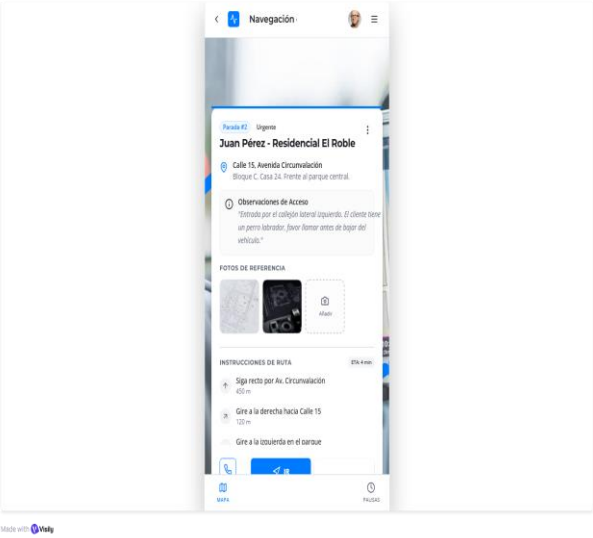
i. Elaboración de dos versiones más de prototipos
   a. Se crearon las siguientes pantallas basados en las historias de usuario de Técnico de campo y de Supervisor Operativo, es importante mencionar que son ideas que servirán para luego crear algo parecido.

Técnico de campo

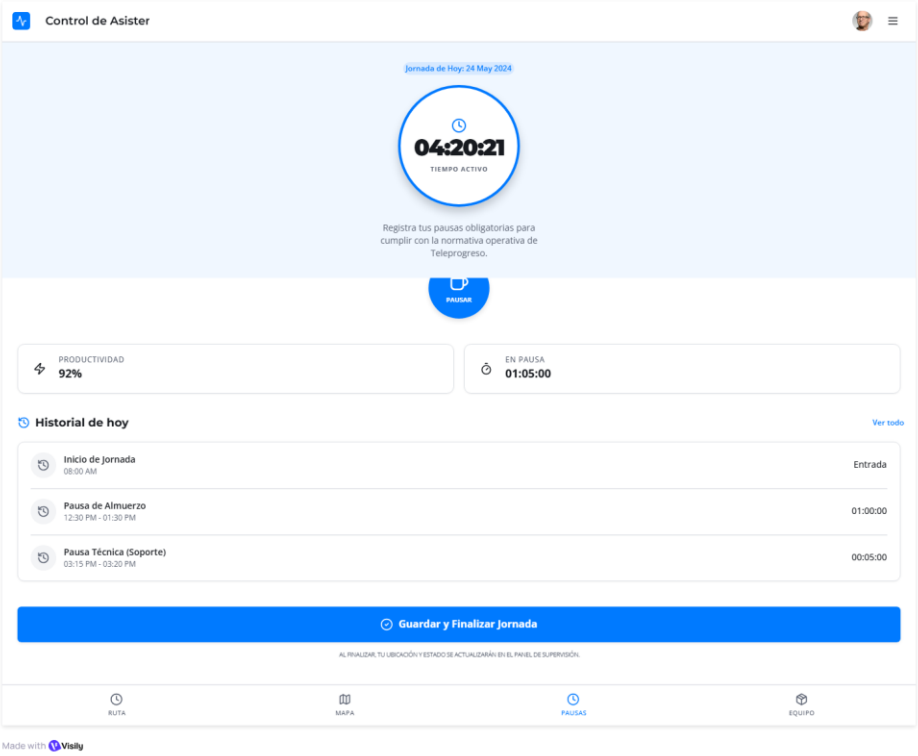
Según lo hablado en las historias de usuario, para este usuario se crearon 4 pantallas nuevas según sus requerimientos y necesidades



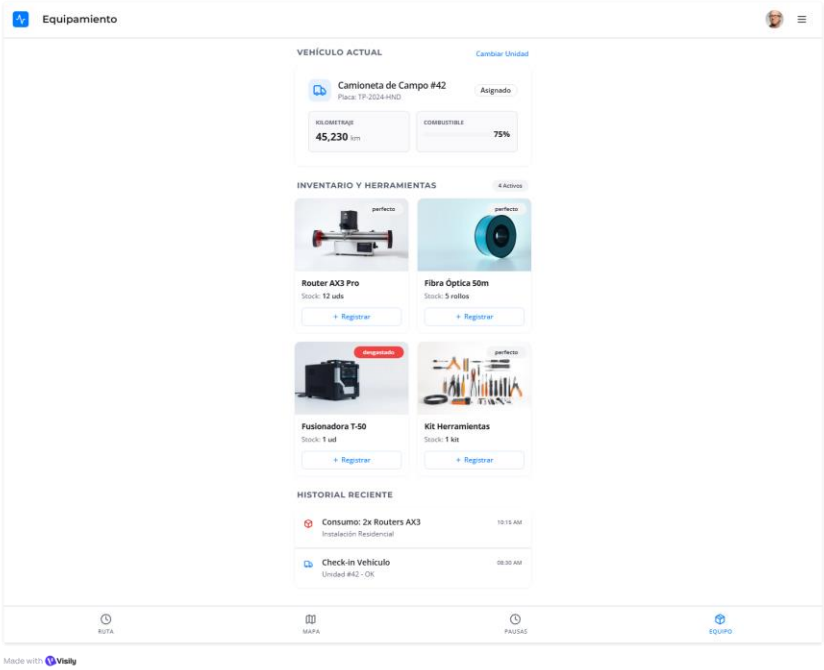
Pantalla 1. Mi ruta diaria



Pantalla 2. Mapa de Ruta

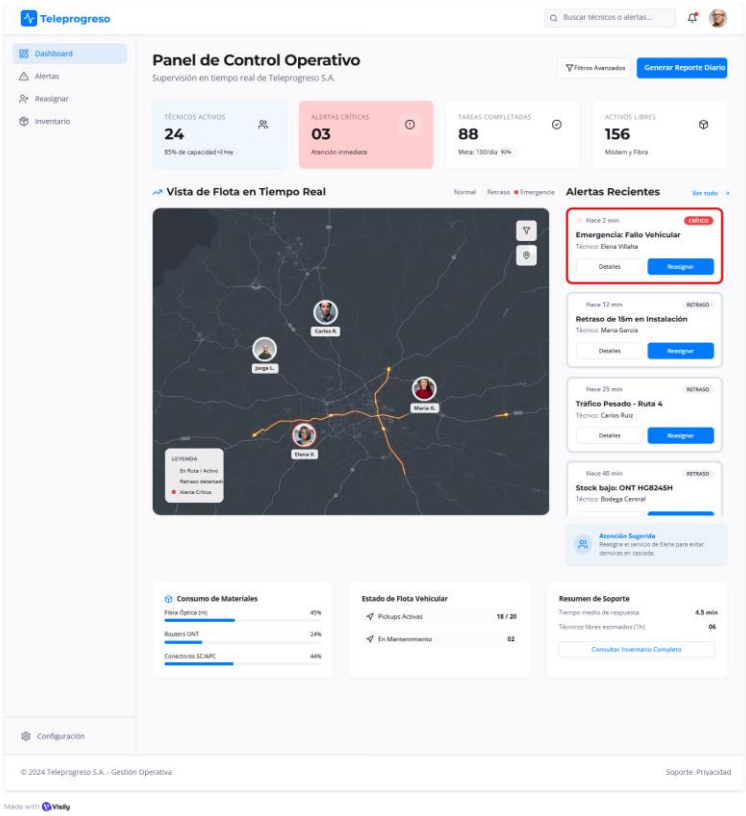


Pantalla 3. Pausas y Asistencia

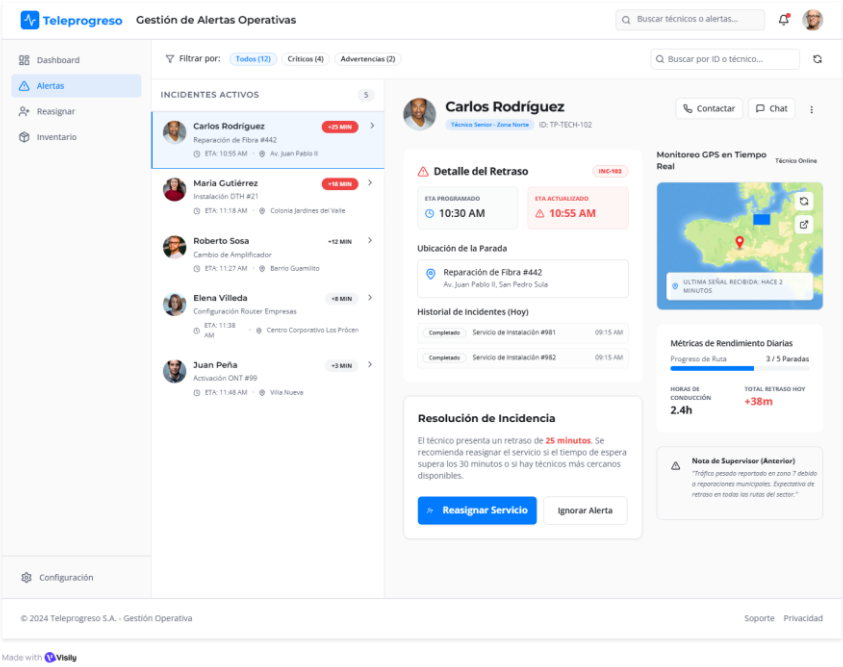


Pantalla 4. inventario y vehículo

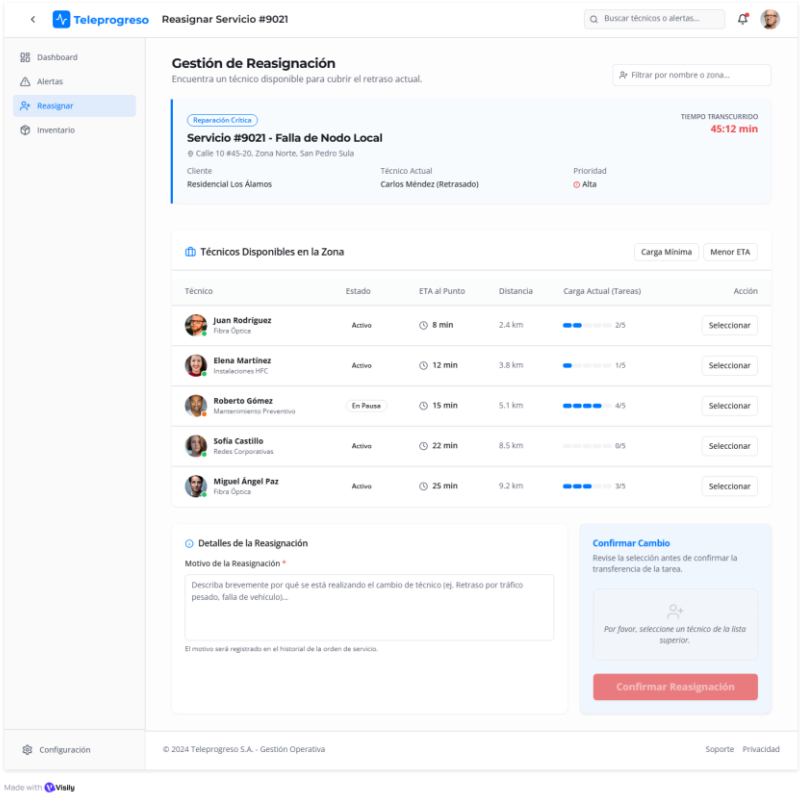
Supervisor Operativo



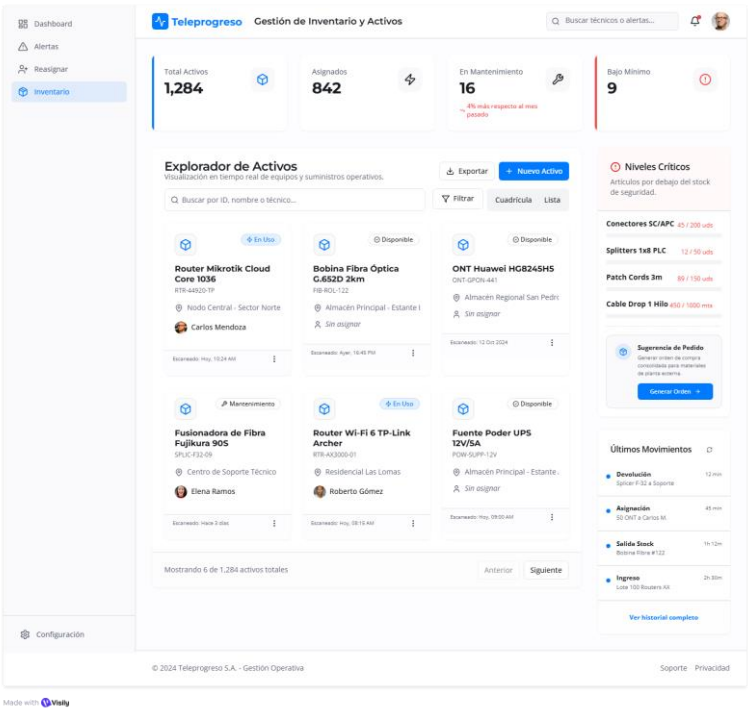
Pantalla 1. Dashboard del Supervisor



Pantalla 2. Alertas y Retrasos

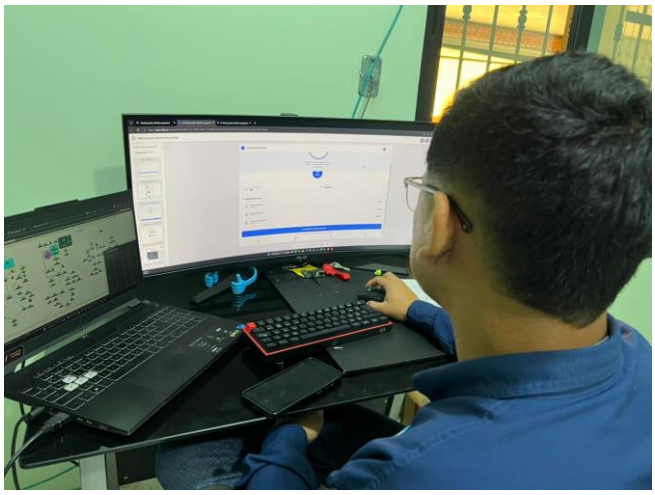


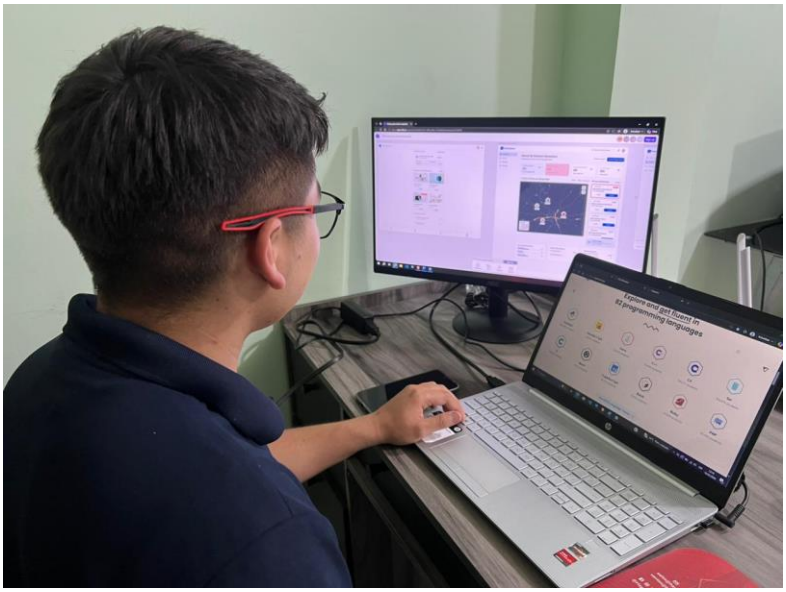
Pantalla 3. Reasignar servicios

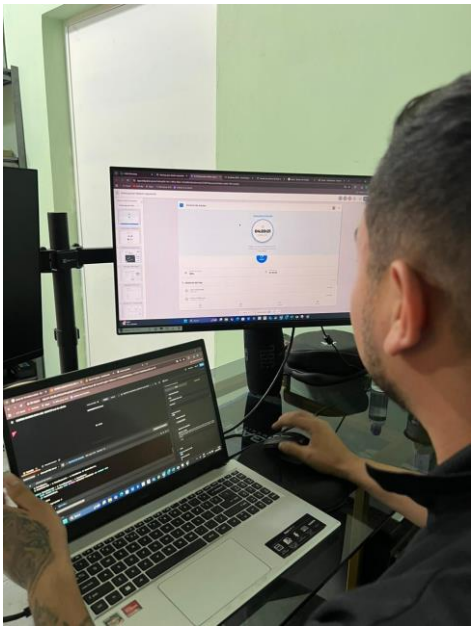


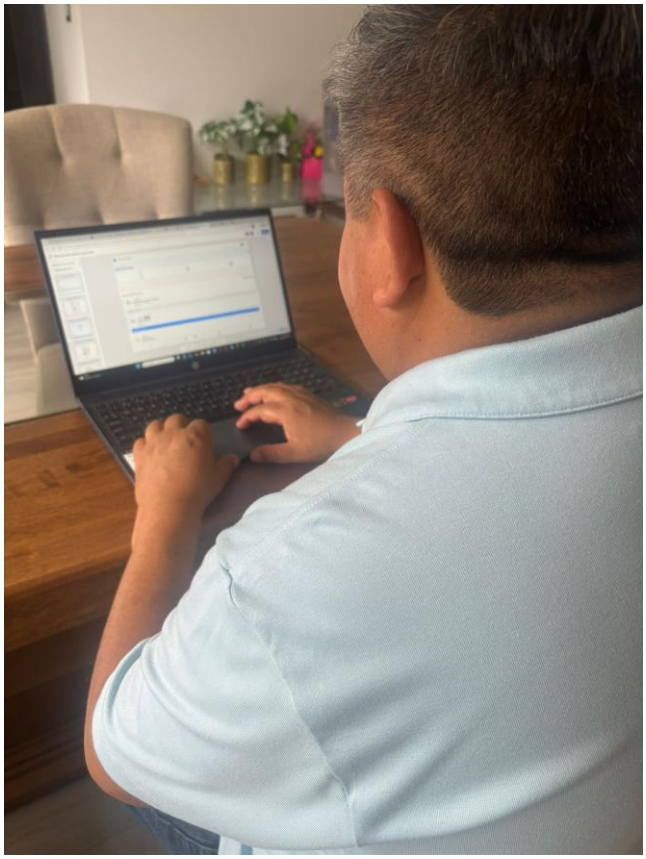
Pantalla 4. Activos en tiempo Real

ii. Mostrar evidencias de cada versión de prototipo. Esto incluye evidencias de que se le mostró a posibles usuarios. (Fotos y/o videos)




Experience and test Liquid in
IoT programming languages





iii. Elaborar una lista de cambios hechos a cada versión de prototipo de acuerdo a observaciones hechas por futuros usuarios entrevistados.

Prototipo 1: Aplicación del Técnico de Campo

Pantalla Correspondiente	**Observación del Usuario (Feedback del Técnico)**	**Cambio Realizado en el Prototipo Final**
Pantalla 1: Ruta Diaria	"A veces olvido visitar algunos lugares ya que donde lo escribo se me borra o doy más vueltas de lo necesario"	Se simplificó la vista ya que de una vez le dice que es lo que tiene que hacer, ósea como primero información general de las paradas y ya luego le muestra un poco de los detalles.

Pantalla 2: mapa de ruta	"No siempre sé cuál es la siguiente parada prioritaria o si hay alguna observación especial de la oficina en el mapa."	Se rediseñó la vista principal para incluir una como ese apartado de urgente, es decir que se tendrá como una lista jerárquica de tareas, donde la prioridad se resalta visualmente y se añadieron precisamente algunas observaciones de acceso para que vaya preparado para cualquier situación.
Pantalla 3: pausas y asistencia	"Me preocupa a veces como perder la noción de cuánto tiempo es que no he aprovechado del todo por cada tarea."	Se añadió una interfaz donde le permita al user realmente el poder como pausar su jornada por x o y motivo, así como un historial para que realmente pueda ver que tan bien ha aprovechado su tiempo.
Pantalla 4: Inventario y Vehículo	"Escribir el nombre de cada herramienta es muy lento cuando estamos cargando el vehículo."	Se eliminó el campo de texto libre y se implementó como un Checklist (lista de selección múltiple) con los activos más frecuentes, permitiendo al técnico registrar todo el equipo mucho más rápido y eficiente.

Prototipo 2: Dashboard del Supervisor Operativo

Pantalla Correspondiente	**Observación del Usuario (Feedback del Supervisor)**	**Cambio/Mejora Realizada en el Prototipo Final**
Pantalla 1: Dashboard Principal	"Necesito ver el resumen del día nada más entrar,	Se añadieron tarjetas de métricas rápidas (técnicos activos, tareas completadas) en la pantalla de inicio del dashboard.

	sin tener que navegar por muchos menús."	
Pantalla 2: Alertas y Retrasos	"Es difícil saber quién es cada técnico en el mapa si solo veo puntos cualquiera."	Se integraron etiquetas con nombres e íconos de estado sobre la persona y además el mapa para identificar al personal al instante.
Pantalla 3: Reasignar servicio	"Si todos los técnicos estuvieran en la zona a quien se llamaría o que.."	Se implementó una jerarquía visual por La carga actual de tareas o trabajo que tiene cada técnico para considerar eso (a quien asignar).
Pantalla 4: Activos en Tiempo Real	"El poder buscar un artículo en una larga lista genera que se pierda mucho tiempo "	Se añadió un diseño y vista que ayuda a visualizar de manera mucho más rápido los activos de la empresa.

iv. Cambios a la lista de requisitos funcionales, si hubiere, de acuerdo a las observaciones hechas a los prototipos.

Durante la validación de los prototipos con los usuarios potenciales que son los técnicos de campo y supervisor operativo, se identificaron algunos ajustes necesarios en los requisitos funcionales del sistema. Estos cambios se realizaron con el objetivo de mejorar la usabilidad, eficiencia y claridad de la plataforma.

Requisito Original	Cambio Realizado	Justificación
El sistema permitirá registrar herramientas mediante entrada de texto.	Se cambió a un sistema de checklist de herramientas predefinidas.	Los técnicos indicaron que escribir cada herramienta era lento durante la carga del vehículo.
El mapa mostrará la ubicación de técnicos mediante puntos.	Se añadieron etiquetas con nombre e iconos de estado.	El supervisor indicó que era difícil identificar quién era cada técnico.
El sistema mostrará las tareas del día.	Se añadió un sistema de prioridades visuales para tareas urgentes.	Los técnicos necesitaban identificar rápidamente la siguiente parada prioritaria.

El sistema permitirá ver los activos de la empresa en lista.	Se agregó una visualización más rápida con filtros o categorización.	El supervisor indicó que buscar en listas largas consume demasiado tiempo.

Finalmente, estos cambios permitieron mejorar la claridad del sistema y adaptarlo mejor a las necesidades reales de los usuarios.

Análisis

Requisitos funcionales del sistema

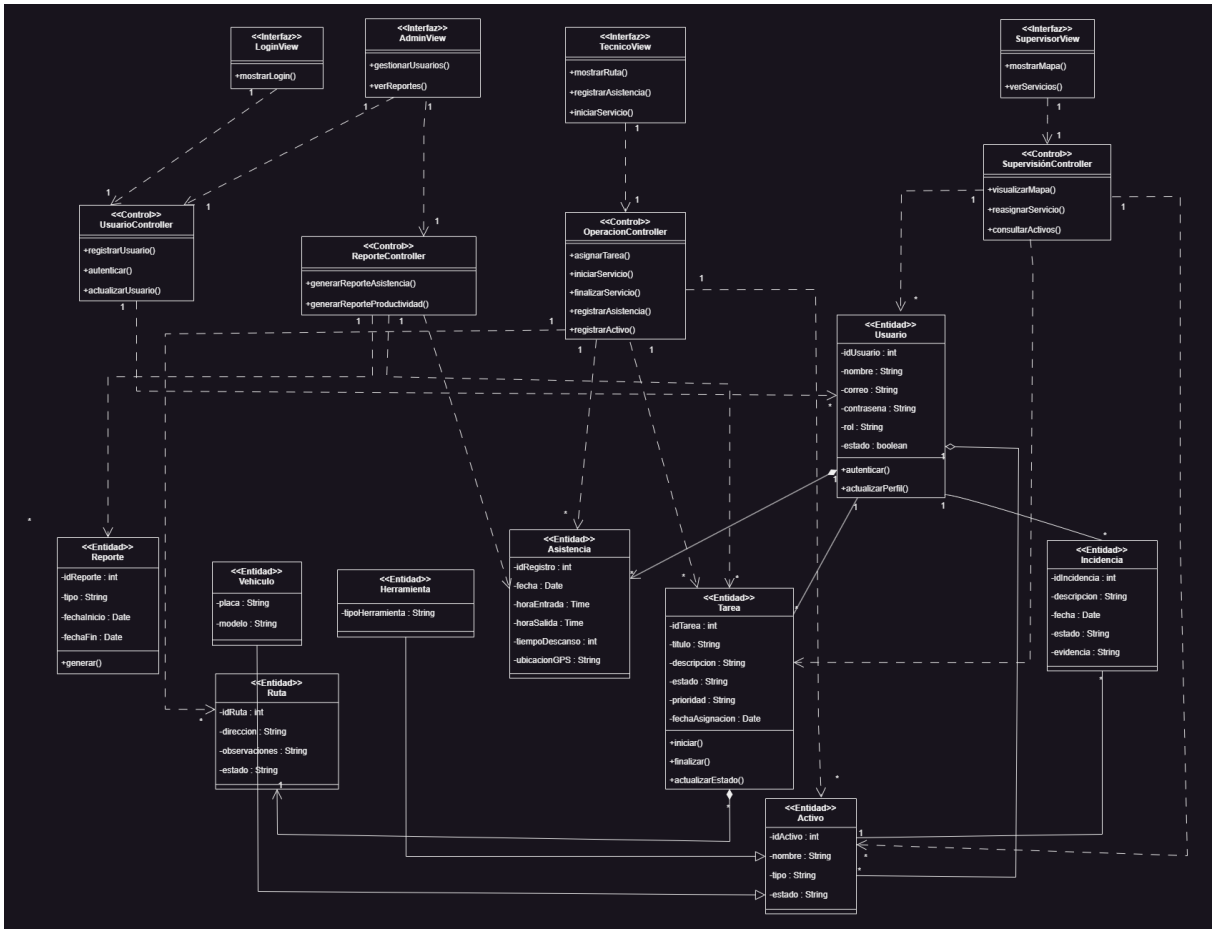
A continuación, se presentan los requisitos funcionales identificados para el sistema. Cada requisito funcional se encuentra vinculado a una historia de usuario previamente definida.

Requisito funcional	Historia de usuario asociada
El sistema deberá permitir que los técnicos registren su entrada laboral utilizando geolocalización.	Como técnico de campo, quiero registrar mi entrada con geolocalización para validar mi jornada laboral.
El sistema deberá permitir que los técnicos registren su salida laboral al finalizar la jornada.	Como técnico de campo, quiero registrar mi salida para dejar constancia del final de mi jornada laboral.
El sistema deberá permitir que los técnicos registren tiempos de pausa o descanso durante la jornada.	Como técnico de campo, quiero registrar tiempos de descanso dentro del sistema para que mi asistencia refleje mi jornada real.
El sistema deberá permitir que los técnicos visualicen su ruta diaria con direcciones y observaciones asignadas.	Como técnico de campo, quiero visualizar mi ruta diaria con direcciones exactas y observaciones para organizar mejor mi jornada.
El sistema deberá permitir que los técnicos marquen el inicio de un servicio asignado.	Como técnico de campo, quiero marcar el inicio de cada servicio para dejar constancia del trabajo que estoy realizando.
El sistema deberá permitir que los técnicos marquen la finalización de un servicio.	Como técnico de campo, quiero marcar la finalización de cada servicio para registrar que el trabajo fue completado.
El sistema deberá permitir que los técnicos registren el uso de vehículos asignados al iniciar la jornada.	Como técnico de campo, quiero registrar el uso del vehículo asignado para mantener control del activo de la empresa.
El sistema deberá permitir que los técnicos registren el uso de herramientas de trabajo mediante un checklist de activos.	Como técnico de campo, quiero registrar el uso de herramientas específicas para llevar control detallado de los activos utilizados.
El sistema deberá permitir que los técnicos reporten incidencias o daños en herramientas o vehículos, incluyendo descripción y evidencia.	Como técnico de campo, quiero reportar daños en herramientas o vehículos con fotos y descripción para dejar evidencia clara del incidente.

El sistema deberá permitir a los supervisores visualizar en un mapa la ubicación en tiempo real de técnicos y vehículos.	Como supervisor operativo, quiero visualizar en un mapa la ubicación de técnicos y vehículos para tener control en tiempo real.
El sistema deberá permitir a los supervisores consultar el estado actual de los servicios asignados.	Como supervisor operativo, quiero ver el estado de cada servicio asignado para identificar retrasos o problemas operativos.
El sistema deberá permitir a los supervisores recibir alertas automáticas cuando un servicio presente retrasos o incidencias.	Como supervisor operativo, quiero recibir alertas automáticas cuando un técnico se retrase en una ruta para actuar en el momento.
El sistema deberá permitir a los supervisores reasignar servicios o rutas entre técnicos disponibles.	Como supervisor operativo, quiero reasignar rutas o servicios desde el sistema para responder a emergencias sin depender de llamadas.
El sistema deberá permitir a los supervisores visualizar la disponibilidad de activos en tiempo real.	Como supervisor operativo, quiero visualizar la disponibilidad de activos para optimizar la asignación de recursos.
El sistema deberá permitir a los gerentes consultar reportes consolidados de asistencia del personal.	Como gerente, quiero visualizar reportes de asistencia consolidados para facilitar el cálculo de nómina.
El sistema deberá permitir a los gerentes consultar reportes históricos de rutas y servicios realizados.	Como gerente, quiero consultar reportes históricos de rutas y servicios para analizar la productividad del personal.
El sistema deberá permitir a la gerencia visualizar indicadores de desempeño del personal y uso de activos.	Como gerente general, quiero acceder a indicadores clave de desempeño para apoyar decisiones estratégicas.
El sistema deberá permitir a los administradores crear cuentas de usuario con distintos roles del sistema.	Como administrador, quiero crear cuentas de usuario con distintos perfiles para controlar los accesos al sistema.
El sistema deberá permitir a los administradores editar o desactivar cuentas de usuario existentes.	Como administrador, quiero gestionar cuentas de usuario para mantener actualizado el acceso al sistema.
El sistema deberá registrar automáticamente historial de cambios y actividades realizadas dentro del sistema.	Como supervisor, quiero consultar el historial de cambios de las tareas para comprender el contexto de cada actividad.

Clases preliminares

Diagrama de clases



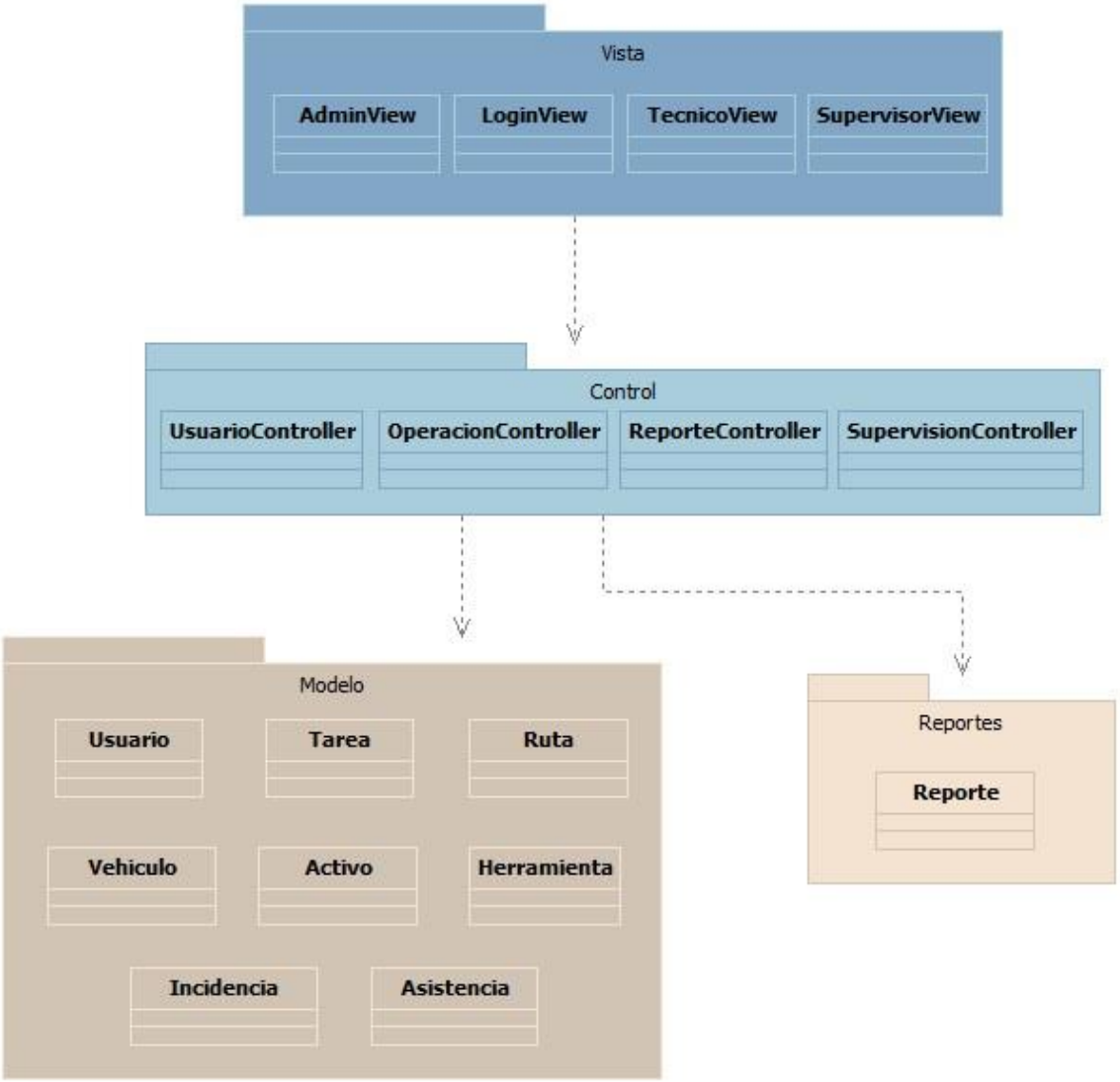
Enlace al diagrama de clases para mejor visualización: https://github.com/Gotkissss/Proyecto-Software-Teleprogreso/blob/main/Corte-3/Diagrama%20de%20clases.drawio

Descripción de las clases

Clase	Tipo	Descripción
LoginView	Interfaz	Interfaz encargada de mostrar la pantalla de inicio de sesión del sistema, permitiendo que el usuario ingrese sus credenciales para autenticarse.
AdminView	Interfaz	Interfaz utilizada por el administrador para gestionar usuarios del sistema y visualizar reportes generados por la plataforma.
TecnicoView	Interfaz	Interfaz utilizada por los técnicos de campo para visualizar su ruta de trabajo, registrar asistencia y marcar el inicio de los servicios asignados.
SupervisorView	Interfaz	Interfaz utilizada por los supervisores para visualizar el mapa de técnicos en campo y monitorear el estado de los servicios.
UsuarioController	Control	Controlador encargado de gestionar las operaciones relacionadas con los usuarios, como registro, autenticación y actualización de información.

ReporteController	Control	Controlador responsable de generar reportes del sistema, como reportes de asistencia y productividad de los técnicos.
OperacionController	Control	Controlador encargado de gestionar las operaciones del sistema relacionadas con las tareas, el registro de asistencia y el uso de activos.
SupervisionController	Control	Controlador encargado de supervisar la operación del sistema, permitiendo visualizar mapas, reasignar servicios y consultar activos disponibles.
Usuario	Entidad	Representa a los usuarios del sistema, incluyendo técnicos, supervisores y administradores, almacenando su información personal y credenciales de acceso.
Tarea	Entidad	Representa un servicio o trabajo asignado a un técnico, incluyendo información como descripción, estado, prioridad y fecha de asignación.
Asistencia	Entidad	Representa el registro de asistencia de un técnico, incluyendo fecha, hora de entrada, hora de salida, tiempos de descanso y ubicación registrada.
Activo	Entidad	Representa los activos de la empresa que pueden ser utilizados durante las operaciones, como vehículos o herramientas.
Vehiculo	Entidad	Representa un tipo específico de activo correspondiente a vehículos utilizados por los técnicos para desplazarse a los servicios.
Herramienta	Entidad	Representa un tipo específico de activo correspondiente a herramientas utilizadas por los técnicos para realizar sus tareas.
Ruta	Entidad	Representa la ubicación o dirección donde se debe realizar una tarea o servicio asignado a un técnico.
Incidencia	Entidad	Representa un reporte de problema o daño relacionado con un activo o con una tarea realizada durante la operación.
Reporte	Entidad	Representa los reportes generados por el sistema para análisis administrativo, como reportes de asistencia o productividad.

Diagrama de paquetes



Descripción de cada paquete y sus componentes

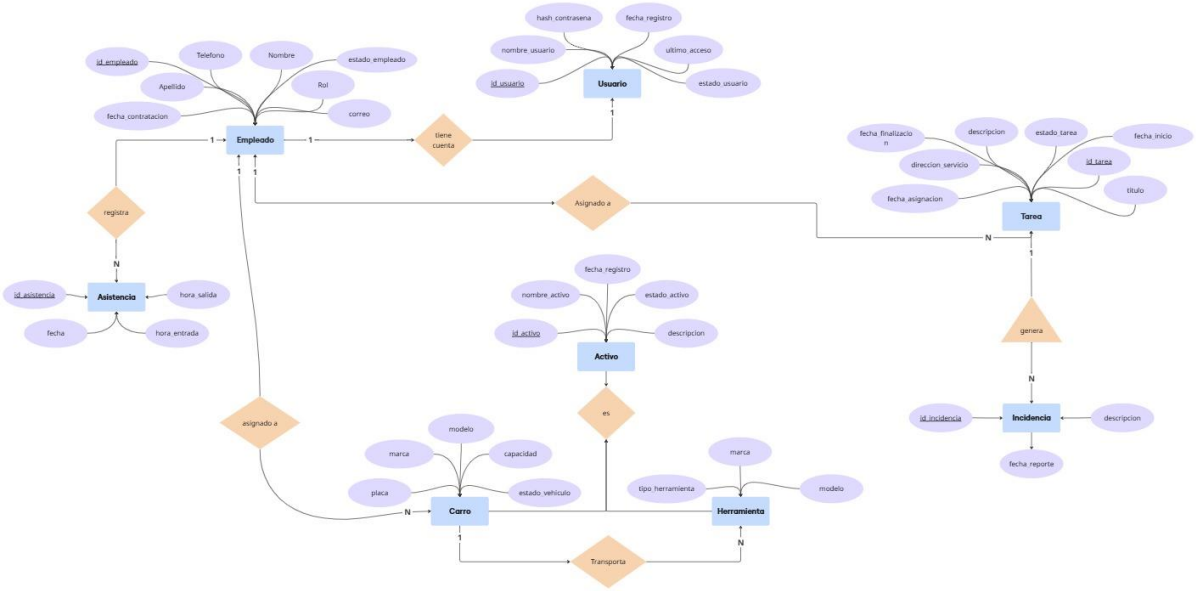
Paquete	Componentes	Descripción
Vista	AdminView, LoginView, TecnicoView, SupervisorView	Contiene las interfaces del sistema encargadas de la interacción directa con los usuarios. Cada vista presenta información y permite ejecutar acciones específicas según el rol del usuario (administrador, técnico o supervisor). Este paquete se comunica únicamente con los controladores para solicitar operaciones del sistema.
Control	UsuarioController, OperacionController, ReporteController, SupervisionController	Contiene los controladores responsables de gestionar la lógica de interacción entre las vistas y el modelo. Los controladores reciben solicitudes de las interfaces, procesan las operaciones necesarias y coordinan el acceso a las entidades del sistema o a los reportes.

Modelo	Usuario, Tarea, Ruta, Vehiculo, Activo, Herramienta, Incidencia, Asistencia	Contiene las entidades del dominio que representan la información principal del sistema. Estas clases almacenan los datos y el estado del sistema relacionados con usuarios, tareas asignadas, activos de la empresa, rutas de trabajo, incidencias y registros de asistencia.
Reportes	Reporte	Contiene la clase responsable de generar reportes administrativos del sistema, como reportes de asistencia o productividad. Esta clase utiliza información del modelo para generar reportes que pueden ser consultados por administradores o supervisores.

Persistencia de datos

Diagrama de clases persistentes

Diagrama Entidad Relación



Diseño

1. Estimación de las dimensiones del sistema

Antes de seleccionar la las tecnologías, se estimó la dimensión que podría alcanzar el sistema, considerando el dominio de la problemática: gestión operativa de técnicos de campo, control de tareas, asistencia y monitoreo de recursos de la empresa Teleprogreso S.A.

Cantidad de usuarios posibles

Actualmente la empresa cuenta con aproximadamente **24 empleados**, entre técnicos de campo, supervisores y personal administrativo.

Sin embargo, considerando el crecimiento de la empresa y la posible expansión de operaciones, el sistema debe diseñarse para soportar un mayor número de usuarios.

Se estima que el sistema podría llegar a manejar aproximadamente:

- **30 – 50 usuarios registrados** en el sistema.
- Distribuidos en:
    - Técnicos de campo
    - Supervisores
    - Administradores
    - Personal administrativo

Cantidad de usuarios simultáneos

Se estima que el sistema podría tener:

- **10 – 20 usuarios concurrentes** en horas de operación.

Esto ocurre principalmente porque:

- los técnicos estarán actualizando estados de tareas.
- los supervisores estarán monitoreando el sistema.
- la administración estará consultando información.

Tasa de respuesta

- El sistema debe cumplir con tiempos de respuesta rápidos: Inicio de sesión: < 4 segundos
- Carga de información (tareas, mapa, asistencia): < 3 segundos
- Registro de acciones (marcar asistencia, actualizar estado): < 3 segundos.

De este modo el sistema será percibido como eficiente y rápido para los usuarios.

Escalabilidad del sistema:

El sistema almacenará distintos tipos de información:

- Usuarios
- Tareas asignadas
- Registros de asistencia
- Incidencias reportadas
- Historial
- Información de activos (vehículos y herramientas)
- Ubicaciones registradas

Tipo de dato	**Estimación anual**
Usuarios	50 registros
Tareas	5000 – 10000 registros
Registros de asistencia	8000 – 12000
Incidencias	1000 – 2000
Historial de cambios	10000 – 20000

Considerando esta información, el sistema podría generar entre 250 MB – 500 MB de datos por trimestre. Lo que permite que el sistema funcione durante varios años incluso con un crecimiento sostenido.

Disponibilidad de la información

El sistema debe estar disponible durante la jornada laboral de la empresa, ya que los técnicos dependen de él para consultar y actualizar sus tareas.

Se establece como requisito:

Disponibilidad mínima del 99% durante horario operativo.

Esto implica que:

- El sistema debe minimizar interrupciones.
- Las tareas y registros deben poder guardarse correctamente.
- La información debe poder consultarse en cualquier momento durante la jornada..

Confiabilidad y Tolerancia a fallos

Para garantizar la confiabilidad del sistema, se consideran las siguientes características:

- Almacenamiento persistente de datos.
- Control de errores en el backend.
- Validación de datos antes de almacenarlos.
- Uso de un gestor de base de datos robusto.

Además, el sistema debe contemplar:

- Replicación o respaldos periódicos de la base de datos.
- Mecanismos de recuperación ante fallos.

2. Selección de tecnologías de desarrollo

Para el desarrollo del sistema se analizaron diferentes tecnologías considerando:

- Rendimiento
- Escalabilidad
- Facilidad de desarrollo
- Compatibilidad con arquitecturas web modernas
- Capacidad de integración con aplicaciones móviles tipo PWA

Arquitectura general del sistema

El sistema se implementará utilizando una arquitectura **cliente-servidor en tres capas**:

1. **Capa de presentación**
2. **Capa de lógica de negocio**
3. **Capa de persistencia de datos**

La aplicación será una **Web Application con soporte PWA**, permitiendo su uso tanto desde navegadores como desde dispositivos móviles.

3. Tecnologías Consideradas

Opción 1: Java + Spring Boot

Descripción

Spring Boot es un framework del lenguaje Java utilizado para construir aplicaciones web empresariales y APIs REST robustas.

Ventajas:

- Alto rendimiento
- Amplio ecosistema empresarial
- Gran capacidad de escalabilidad
- Excelente manejo de seguridad y transacciones
- Muy utilizado en sistemas corporativos

Desventajas:

- Mayor complejidad de configuración.
- Curva de aprendizaje más alta.
- Mayor cantidad de código necesario para implementar funcionalidades simples.

Debido a su complejidad y al alcance del proyecto, esta opción se consideró menos adecuada para el desarrollo inicial del sistema.

Opción 2: Node.js + Express

Descripción

Node.js permite ejecutar JavaScript en el servidor, mientras que Express es un framework ligero para la construcción de APIs y aplicaciones web.

Ventajas:

- Uso del mismo lenguaje en frontend y backend.
- Gran cantidad de librerías disponibles.
- Alto rendimiento para aplicaciones web.

Desventajas:

- Puede volverse difícil de mantener en proyectos grandes si no se estructura adecuadamente.
- Manejo de concurrencia y arquitectura puede requerir mayor disciplina de diseño.

Aunque es una tecnología muy popular, se decidió evaluar otras alternativas que facilitaran la construcción de APIs de forma más estructurada.

Opción 3: Python + FastAPI (Tecnología seleccionada)

Descripción

FastAPI es un framework moderno para el desarrollo de APIs web utilizando el lenguaje Python. Está diseñado para ser rápido, fácil de usar y altamente eficiente.

Ventajas:

- Alto rendimiento comparable con frameworks modernos
- Desarrollo rápido de APIs REST
- Documentación automática de endpoints
- Integración sencilla con bases de datos
- Código claro y mantenible
- Validación automática de datos

Desventajas:

- Ecosistema más reciente que otros frameworks tradicionales.
- Algunas herramientas empresariales aún están en desarrollo.

Justificación

Se seleccionó FastAPI porque permite desarrollar APIs robustas y eficientes con una arquitectura clara, facilitando la comunicación entre el frontend y el backend.

Además, su capacidad para manejar múltiples solicitudes concurrentes y su integración sencilla con bases de datos lo convierten en una opción adecuada para sistemas que gestionan tareas en tiempo real.

También facilita la creación de servicios REST necesarios para la comunicación con la Web App y la PWA.

4. Tecnologías de frontend

React

Descripción

React es una biblioteca de JavaScript utilizada para construir interfaces de usuario dinámicas basadas en componentes reutilizables.

Ventajas:

- Desarrollo modular basado en componentes
- Alto rendimiento mediante renderizado eficiente
- Gran comunidad y ecosistema
- Ideal para aplicaciones web interactivas
- Facilita la implementación de interfaces complejas como dashboards o mapas

Desventajas:

- Requiere configuración inicial
- Puede necesitar librerías adicionales para ciertas funcionalidades

Justificación

React permite construir una interfaz dinámica capaz de manejar:

- Visualización de tareas
- Mapas de ubicación
- Paneles administrativos
- Actualizaciones en tiempo real

Además, facilita la implementación de funcionalidades PWA.

5. Tecnologías PWA

El sistema se diseñará como una Progressive Web App (PWA).

Esto permitirá:

- instalar la aplicación en dispositivos móviles
- trabajar desde navegador o desde acceso directo
- mejorar la experiencia de usuario para técnicos de campo

Ventajas:

- no requiere publicar en tiendas de aplicaciones
- funciona en dispositivos móviles
- puede almacenar datos localmente
- permite notificaciones y acceso rápido

Desventajas:

- algunas capacidades del hardware pueden ser más limitadas que en apps nativas
- dependencia del navegador del dispositivo

Aun así, es una solución eficiente para aplicaciones empresariales internas.

6. Selección del gestor de base de datos

Para el almacenamiento persistente de la información del sistema se evaluaron distintos gestores de bases de datos relacionales. La selección se realizó considerando criterios como:

- integridad de los datos
- manejo de múltiples usuarios concurrentes
- escalabilidad del sistema
- soporte para transacciones
- mecanismos de respaldo y recuperación

Los gestores considerados fueron PostgreSQL, MySQL y SQLite.

PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto ampliamente utilizado en aplicaciones empresariales y sistemas que requieren alta confiabilidad y consistencia de datos.

Ventajas:

- Alta confiabilidad en el manejo de datos.
- Soporte completo para **transacciones ACID**, lo que garantiza la integridad de la información.
- Excelente manejo de **usuarios concurrentes**, lo cual es importante para sistemas con múltiples accesos simultáneos.
- Capacidad de **escalabilidad**, permitiendo manejar grandes volúmenes de información.
- Soporte para **replicación, respaldo y recuperación de datos**.
- Amplio soporte para integrarse con frameworks modernos como FastAPI.

Desventajas:

- Puede requerir mayor configuración inicial en comparación con gestores más simples.
- La administración puede ser más compleja que en soluciones ligeras como SQLite.

MySQL

MySQL es uno de los sistemas de gestión de bases de datos más populares y ampliamente utilizados en aplicaciones web.

Ventajas:

- Amplia adopción en la industria.
- Buen rendimiento para aplicaciones web.
- Gran cantidad de herramientas y documentación disponibles.
- Integración sencilla con múltiples tecnologías y lenguajes de programación.

Desventajas:

- Algunas funcionalidades avanzadas requieren configuraciones adicionales.
- En ciertos casos, el manejo avanzado de transacciones y características complejas puede ser más limitado en comparación con PostgreSQL.

SQLite

SQLite es un gestor de bases de datos ligero que funciona de manera embebida dentro de la aplicación, sin necesidad de un servidor de base de datos separado.

Ventajas:

- Muy ligero y fácil de configurar.
- No requiere instalación ni configuración de servidor.
- Adecuado para aplicaciones pequeñas o prototipos.

Desventajas:

- No está diseñado para manejar múltiples usuarios concurrentes de manera eficiente.
- Escalabilidad limitada para aplicaciones con gran volumen de datos.
- Carece de algunas características avanzadas necesarias para sistemas empresariales.

Selección final

Después de analizar las diferentes alternativas, se seleccionó PostgreSQL como gestor de base de datos para el sistema.

La decisión se tomó considerando que el sistema requiere:

- manejo confiable de datos operativos críticos (tareas, asistencia, incidencias, usuarios).
- soporte para múltiples usuarios concurrentes.
- transacciones seguras.
- posibilidad de crecimiento del sistema en el futuro.

PostgreSQL ofrece un equilibrio adecuado entre rendimiento, confiabilidad y escalabilidad, lo que lo convierte en una opción apropiada para el desarrollo de este sistema.

Además, proporciona herramientas robustas para:

- respaldo de información
- replicación de datos
- mantenimiento de integridad de la base de datos

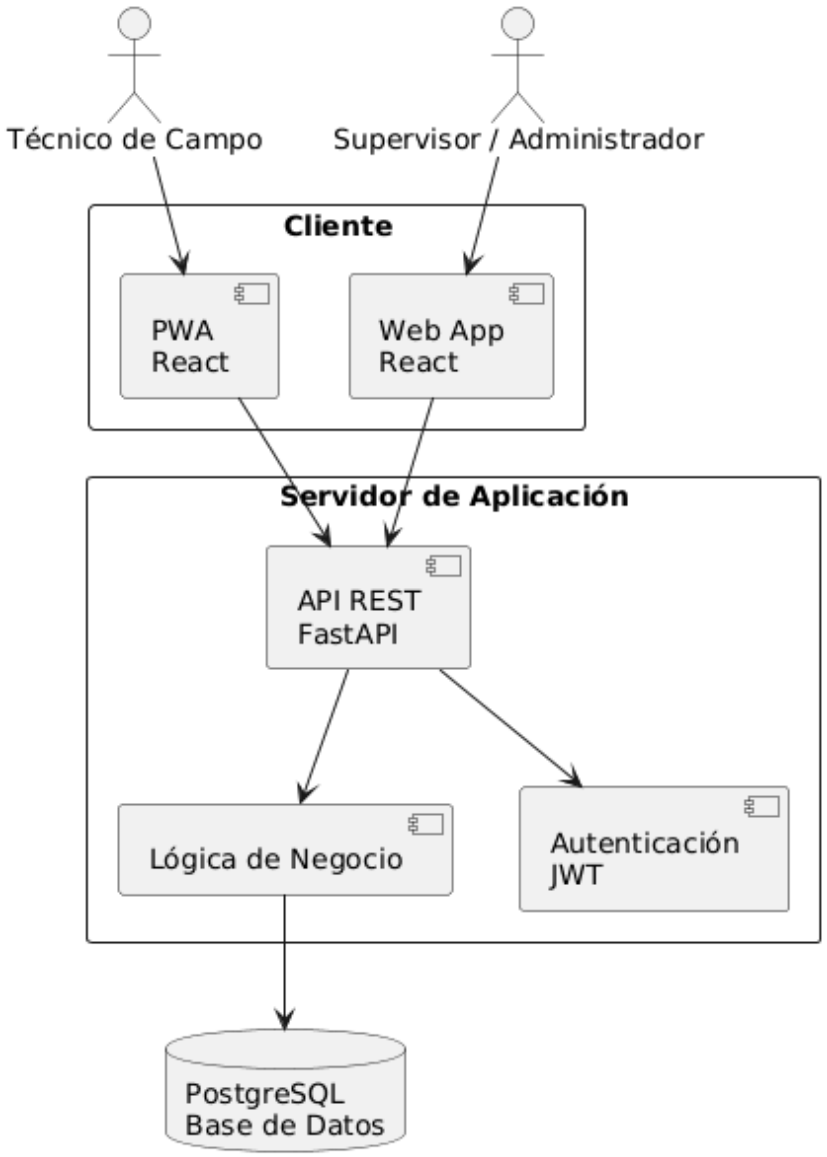
Por estas razones, PostgreSQL fue seleccionado como la tecnología de almacenamiento persistente para el sistema.

1. Diagrama de Arquitectura del Sistema

Este diagrama muestra la arquitectura de alto nivel, es decir, cómo interactúan los usuarios, el frontend, el backend y la base de datos.

El sistema se implementa utilizando una arquitectura cliente-servidor de tres capas:

- Presentación
- Lógica de negocio
- Persistencia de datos



Explicación

Los técnicos de campo acceden al sistema desde dispositivos móviles mediante una Progressive Web App (PWA).
Los administradores y supervisores utilizan la aplicación web desde navegadores en computadoras.

Ambos clientes se comunican con el backend a través de peticiones HTTP hacia una API REST implementada en FastAPI.

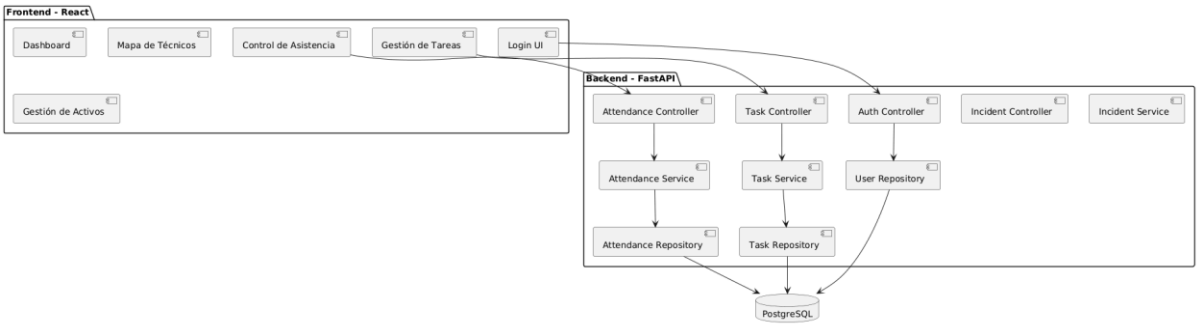
El backend se encarga de:

- Autenticación
- Lógica de negocio
- Procesamiento de datos
- Comunicación con la base de datos

Finalmente, toda la información se almacena de forma persistente en una base de datos PostgreSQL.

2. Diagrama de Componentes

Este diagrama muestra cómo se divide internamente el backend y frontend del sistema.



Explicación

El sistema está compuesto por tres subsistemas principales:

Frontend

Aplicación desarrollada con React que contiene módulos para:

- Autenticación
- Gestión de tareas
- Control de asistencia
- Visualización de mapa
- Gestión de activos

Backend

El backend está organizado en módulos funcionales:

- Controladores (API endpoints)
- Servicios (lógica de negocio)
- Repositorios (acceso a datos)
- Modelos de dominio

Persistencia

La base de datos PostgreSQL almacena todas las entidades del sistema.

3. Arquitectura con Patrones de Diseño

Para garantizar una estructura clara y mantenible del sistema, se utilizará una arquitectura basada en patrones de diseño ampliamente utilizados en aplicaciones web modernas.

Los principales patrones utilizados serán:

- MVC
- Repository
- DTO

Patrón MVC

El patrón **Modelo-Vista-Controlador (MVC)** permite separar las responsabilidades dentro del sistema.

Modelo

Representa las entidades del sistema y su estructura de datos.

Ejemplos:

- Usuario
- Tarea
- Asistencia
- Incidencia
- Activo

Estos modelos se almacenan en la base de datos PostgreSQL.

Vista

La vista corresponde a la interfaz de usuario desarrollada en React.

Permite:

- Visualizar tareas
- Mostrar mapas
- Registrar asistencia
- Consultar información

Controlador

Los controladores son implementados en FastAPI y se encargan de:

- Recibir solicitudes HTTP
- Validar datos
- Llamar a los servicios correspondientes
- Devolver respuestas al cliente.

Patrón Repository

El patrón **Repository** permite separar el acceso a datos de la lógica de negocio.

Cada repositorio se encarga de:

- consultar datos
- insertar registros
- actualizar información
- eliminar registros

Ejemplos de repositorios:

- UserRepository
- TaskRepository
- AttendanceRepository
- IncidentRepository

Esto facilita cambiar la tecnología de base de datos sin afectar el resto del sistema.

Patrón DTO (Data Transfer Object)

Los DTO se utilizan para transferir datos entre el backend y el frontend.

Permiten:

- Evitar exponer directamente las entidades de la base de datos
- Controlar qué información se envía al cliente
- Validar datos de entrada

Ejemplos:

- UserDTO
- TaskDTO
- AttendanceDTO
- IncidentDTO

Los DTO son utilizados principalmente en las respuestas de la API REST.

Planificación e Informe de Gestión

A continuación, se presenta el desglose de tareas asignadas para el desarrollo de este tercer corte del proyecto:

Tarea	Responsable
Elaboración de la hoja de presentación del documento, incluyendo título del proyecto, curso, catedrático e integrantes del equipo.	Joel Nerio
Redacción del resumen del proyecto, describiendo el propósito del sistema, la problemática identificada y los objetivos principales del trabajo.	Joel Nerio
Redacción de la introducción del informe, incluyendo la descripción de la empresa Teleprogreso S.A., su estructura organizacional y los procesos principales de la empresa.	Joel Nerio
Redacción de la descripción de la idea del proyecto y explicación de la problemática que motiva el desarrollo del sistema propuesto.	Joel Nerio
Redacción de los objetivos generales y específicos del informe del tercer corte del proyecto.	Joel Nerio
Elaboración de nuevas versiones de prototipos del sistema basadas en las historias de usuario definidas previamente.	Joel Nerio y Wilson Peña
Documentación de los prototipos desarrollados para los diferentes tipos de usuarios del sistema.	Joel Nerio y Wilson Peña
Recopilación de evidencias de interacción con usuarios durante la validación de los prototipos.	Joel Nerio y Wilson Peña
Análisis de observaciones recibidas durante la validación de los prototipos y registro de mejoras realizadas en el sistema.	Joel Nerio y Wilson Peña
Actualización de requisitos funcionales a partir del feedback recibido durante las pruebas de prototipo.	Joel Nerio y Wilson Peña
Elaboración de la lista de requisitos funcionales del sistema basados en las historias de usuario previamente identificadas.	Biancka Raxón
Vinculación de cada requisito funcional con su respectiva historia de usuario dentro del modelo del sistema.	Biancka Raxón
Elaboración del diagrama de clases preliminar del sistema utilizando el patrón Modelo Vista Controlador (MVC).	Biancka Raxón
Redacción de la descripción detallada de cada clase del sistema.	Biancka Raxón
Elaboración del diagrama de paquetes del sistema manteniendo alta cohesión y bajo acoplamiento.	Biancka Raxón
Redacción de la descripción de cada paquete y de los componentes que contiene.	Biancka Raxón
Elaboración del diagrama de clases persistentes que representan las tablas de la base de datos del sistema.	Biancka Raxón
Elaboración del diagrama entidad-relación de la base de datos relacional utilizada por el sistema.	Biancka Raxón
Estimación de las dimensiones del sistema considerando cantidad de usuarios potenciales, usuarios concurrentes y volumen de datos esperado.	Diego Quixchan, Juan Gualim y Harry Mendez
Estimación de tiempos de respuesta del sistema, disponibilidad y requerimientos de confiabilidad.	Diego Quixchan, Juan Gualim y Harry Mendez

Análisis de tecnologías posibles para el desarrollo del backend del sistema.	Diego Quixchan, Juan Gualim y Harry Mendez
Análisis de tecnologías para el desarrollo del frontend del sistema.	Diego Quixchan, Juan Gualim y Harry Mendez
Comparación de diferentes gestores de bases de datos y análisis de sus ventajas y desventajas.	Diego Quixchan, Juan Gualim y Harry Mendez
Selección de la tecnología de desarrollo y justificación de la decisión tomada por el equipo.	Diego Quixchan, Juan Gualim y Harry Mendez
Diseño de la arquitectura general del sistema basada en una arquitectura cliente-servidor.	Diego Quixchan, Juan Gualim y Harry Mendez
Elaboración del diagrama de arquitectura del sistema.	Diego Quixchan, Juan Gualim y Harry Mendez
Elaboración del diagrama de componentes del sistema.	Diego Quixchan, Juan Gualim y Harry Mendez
Definición de los patrones de diseño utilizados en la arquitectura del sistema (MVC, Repository y DTO).	Diego Quixchan, Juan Gualim y Harry Mendez
Elaboración del desglose de tareas del proyecto para la sección de planificación.	Biancka Raxón
Recopilación de los formularios LOGT de cada integrante del equipo.	Biancka Raxón
Redacción del informe de gestión del tiempo del equipo de desarrollo.	Biancka Raxón
Análisis del rendimiento del equipo de trabajo, identificando aspectos positivos y áreas de mejora durante el desarrollo del corte.	Biancka Raxón

Formularios LOGT individuales

Nombre: Joel Nerio

Carné: 24253

Fecha	Inicio	Fin	Tiempo de Interrupción	Delta de Tiempo	Fase	Comentarios

Nombre: Biancka Raxon

Carné: 24960

Fecha	Inicio	Fin	Tiempo de Interrupción	Delta de Tiempo	Fase	Comentarios

Nombre: Diego Quixchan

Carné: 24903

Fecha	Inicio	Fin	Tiempo de Interrupción	Delta de Tiempo	Fase	Comentarios
03/13/2026	7:00pm	8:00pm	0 min	0	Diseño	Discusión de la tecnología a escoger.
03/16/2026	6:00pm	9:00pm	0 min	0	Diseño	Redacción de todo lo solicitado en las instrucciones.
03/18/2026	9:00am	1:00pm	0 min	0	Presentación	Hice la presentación y preparé el OBS.

Nombre: Harry Mendez

Carné: 24089

Fecha	Inicio	Fin	Tiempo de Interrupción	Delta de Tiempo	Fase	Comentarios
03/13/2026	7:00pm	8:00pm	0 min	0	Diseño	Discusión de la tecnología a escoger.
03/16/2026	6:00pm	11:00 pm	20 min	280 min	Base de Datos.	Diagramas y normalización de la base de datos.
03/15/2026						

Nombre: Juan Gualim

Carné: 24852

Fecha	Inicio	Fin	Tiempo de Interrupción	Delta de Tiempo	Fase	Comentarios

Nombre: Wilson Peña

Carné: 24760

Fecha	Inicio	Fin	Tiempo de Interrupción	Delta de Tiempo	Fase	Comentarios
11/03/2026	8 am	11 am	15 min	165 min	p	

Evaluación del trabajo en equipo y gestión del tiempo